Lecture: Hashing Paradigms and Frequency Arrays

1. The Core Paradigm: Precomputation and Hashing

When processing multiple queries (e.g., Q queries) on a static dataset, repeatedly scanning the entire dataset ($O(N)$ time per query) leads to severe performance degradation ($O(N \times Q)$ overall time). Hashing solves this by trading space for time.

- **Definition:** Hashing is the technique of precomputing and storing data (like frequencies or existences) into an auxiliary data structure so that subsequent fetch operations evaluate in constant time, $O(1)$.
- **Frequency Arrays:** The most primitive form of hashing. It utilizes the actual value of an element as the mathematical index of a new array. If the number `5` appears in the dataset, we increment the value at `hashArray[5]`.
- **State Segmentation:** Every hashing algorithm consists of two strict phases:
  1. **Precompute Phase:** Traversing the data once to populate the hash map/array.
  2. **Fetch Phase:** Answering queries instantly by directly accessing the hash map/array index.

Phase Deconstruction: The Optimization Transition

Approach 1: The Brute Force (Linear Search Counting)

To count how many times a target element appears, the naive approach iterates through the entire array for every single query.

C++

```
int count_occurrences(int n, int arr[], int target) {
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            cnt += 1;
        }
    }
    return cnt;
}
```

Metric	Complexity	Justification
Time Complexity	$O(N \times Q)$	For Q queries, the algorithm runs an $O(N)$ loop every time. If $N = 10^5$ and $Q = 10^5$, total

Metric	Complexity	Justification
		operations = 10^{10} (Fails the 10^8 CP 1-second limit rule).
Space Complexity	$O(1)$	No extra memory allocated.

Approach 2: Optimal Frequency Array Hashing

By allocating an array slightly larger than the maximum possible value in our dataset, we can compute all frequencies simultaneously.

Problem-Solving Deconstruction

Problem 1: Number Hashing (Frequency Array)

1. Problem Statement & Constraints: Given an array of size N containing positive integers (with a known maximum bound, e.g., ≤ 12), answer Q queries. Each query asks for the exact frequency of a given target integer.

2. Core Intuition: Since the maximum element is known and small (e.g., 12), we can allocate a fixed `hash` array of size 13 initialized to 0. We map the data value directly to the index of the `hash` array (`hash[arr[i]]`).

3. Algorithmic Steps:

1. **Initialize:** Declare `int hash[13] = {0};` to track elements from 0 to 12.
2. **Precompute:** Open a `for` loop from $i = 0$ to $N - 1$. Increment the frequency at the specific index: `hash[arr[i]] += 1;`.
3. **Fetch:** Open a `while` loop to process `target` queries (`while(target--)`).
4. Read the query `number`.
5. Immediately `cout << hash[number]` to fetch the result in $O(1)$ time.

4. Dry Run:

Input Array: `arr = [1, 3, 3, 2]`

1. Array initialized: `hash[13] = {0,0,0,0...}`
2. Process `1`: `hash[1]++`. (`hash` becomes `{0,1,0,0...}`)
3. Process `3`: `hash[3]++`. (`hash` becomes `{0,1,0,1...}`)
4. Process `3`: `hash[3]++`. (`hash` becomes `{0,1,0,2...}`)
5. Process `2`: `hash[2]++`. (`hash` becomes `{0,1,1,2...}`)

Query fetching target `3`: Return `hash[3]` which is strictly `2`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N+Q)$	$O(N)$ to iterate and precompute the hash array, plus $O(Q)$ to answer the queries. They run sequentially.
Space Complexity	$O(\text{Max\_Element})$	Requires an auxiliary array of size equal to the maximum possible value in the input data.

C++

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int arr[5] = {1, 3, 2, 1, 3};

    // Phase 1: Precompute (Hashing)
    // Constraint: Max element in array is known to be <= 12
    int hash[13] = {0};
    for (int i = 0; i < n; i++) {
        hash[arr[i]] += 1;
    }

    int queries = 3;
    // Phase 2: Fetching
    while (queries--) {
        int number;
        cin >> number; // e.g., user inputs 3

        // Fetch in O(1)
        cout << "Frequency: " << hash[number] << endl;
    }
    return 0;
}
```

Problem 2: Character Hashing (ASCII Mapping)

1. Problem Statement & Constraints: Given a string S containing strictly uppercase English letters, answer Q queries asking for the frequency of a specific character.

2. Core Intuition: We cannot create an array where the index is a character (`hash['A']`). We must mathematically map the characters to integers. The English alphabet has 26 letters.

- **The ASCII Trick:** Every character has an underlying integer value. `'A'` is 65, `'B'` is 66, etc. If we subtract `'A'` from any uppercase character, it normalizes into a strict 0-based index:

- `'A' - 'A' = 0`
- `'B' - 'A' = 1`
- `'Z' - 'A' = 25`

3. Algorithmic Steps:

1. Initialize an array of size 26 for the alphabet: `int hash[26] = {0};`.
2. Iterate through the string `s` using `i=0` to `s.size()-1`.
3. Normalize the character and increment its frequency: `hash[s[i] - 'A']++`.
4. Process queries by reading a character `c`.
5. Fetch its frequency by normalizing the query character: `cout << hash[c - 'A']`.

4. Dry Run:

Input String: `s = "ABA"`

1. Setup: `hash[26] = {0}`
2. Process `s[0]` ('A'): Index = `'A' - 'A' = 0`. `hash[0]++`.
3. Process `s[1]` ('B'): Index = `'B' - 'A' = 1`. `hash[1]++`.
4. Process `s[2]` ('A'): Index = `'A' - 'A' = 0`. `hash[0]++`.

   Query fetching 'B': Normalizes to `1`. Return `hash[1]` which is `1`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(N+Q)$	String traversal $O(N)$ followed by Q constant-time fetches.
Space Complexity	$O(1)$	Strictly bounded to an array of size 26, regardless of string length. $O(26)$ simplifies to $O(1)$.

C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;
    cin >> s;

    // Phase 1: Precompute (Character Hashing)
    int hash[26] = {0};
    for (int i = 0; i < s.size(); i++) {
        // Normalizes uppercase char to 0-25 index range
```

```
        hash[s[i] - 'A']++;
    }

    int q;
    cin >> q;
    // Phase 2: Fetching
    while (q--) {
        char c;
        cin >> c;

        // Normalize query char and fetch in O(1)
        cout << hash[c - 'A'] << endl;
    }
    return 0;
}
```

Note on Generalization: If the string contains both lowercase, uppercase, and special characters, you must allocate a hash array of size 256 (`int hash[256] = {0};`) and cast the character directly to an integer without subtracting anything (e.g., `hash[s[i]]++`).